Practical No. 1: Mini Generative AI – Next Word Prediction

Aim: To understand the basic concept of Generative Artificial Intelligence by developing a simple next-word prediction model using Python.

Python Program:

In [1]:
```
#Practical 1
import random

# Training sentences
sentences = [
    "I love data science",
    "I love machine learning",
    "data science is amazing",
    "machine learning is powerful",
    "I enjoy learning python"
]

# Dictionary to store word relationships
model = {}

# Build the model
for sentence in sentences:
    words = sentence.split()

    for i in range(len(words) - 1):
        current_word = words[i]
        next_word = words[i + 1]

        if current_word not in model:
            model[current_word] = []

        model[current_word].append(next_word)

print("Learned Model")
print(model)

# User input
word = input("\nEnter starting word: ")

generated_text = word

# Generate text
for i in range(5):

    if word in model:
        word = random.choice(model[word])
```

```
        generated_text = generated_text + " " + word
    else:
        break

print("\nGenerated Text:")
print(generated_text)
```

```
Learned Model
{'I': ['love', 'love', 'enjoy'], 'love': ['data', 'machine'], 'data': ['s
cience', 'science'], 'machine': ['learning', 'learning'], 'science': ['i
s'], 'is': ['amazing', 'powerful'], 'learning': ['is', 'python'], 'enjo
y': ['learning']}

Enter starting word: data

Generated Text:
data science is powerful
```

Practical No. 2: Text Generation and Text Summarization using Generative AI

Aim: To understand the concepts of text generation and text summarization using Python.

Part A: Text Generation

Python Program:

In [2]:
```
#practical 2 part 1
responses = {
    "ai": "Artificial Intelligence is transforming the world.",
    "python": "Python is a popular programming language for AI.",
    "data": "Data is the foundation of machine learning."
}

word = input("Enter a keyword (ai/python/data): ").lower()

if word in responses:
    print("\nGenerated Text:")
    print(responses[word])
else:
    print("No text available for the given keyword.")
```

```
Enter a keyword (ai/python/data): data

Generated Text:
Data is the foundation of machine learning.
```

Part B: Text Summarization using Hugging Face Transformer

Python Program:

In [3]:
```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Load the tokenizer and model directly for summarization
tokenizer = AutoTokenizer.from_pretrained("facebook/bart-large-cnn")
model = AutoModelForSeq2SeqLM.from_pretrained("facebook/bart-large-cnn")

# Input paragraph
text = """
Artificial Intelligence is transforming every industry by automating task
improving decision-making, and generating human-like content.
Generative AI is a branch of AI capable of creating text, images,
music, videos, and even software code.
"""

# Prepare the input for the model
inputs = tokenizer([text], max_length=1024, return_tensors="pt", truncati

# Generate summary
# BART models often use 'generate' with specific parameters for summariza
summary_ids = model.generate(
    inputs["input_ids"],
    num_beams=4,
    min_length=15,
    max_length=40,
    early_stopping=True
)
summary_text = tokenizer.decode(summary_ids[0], skip_special_tokens=True,

# Display original text
print("Original Text:\n")
print(text)
# Display summary
print("\nGenerated Summary:\n")
print(summary_text)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please s
et a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthentica
ted requests to the HF Hub. Please set a HF_TOKEN to enable higher rate l
imits and faster downloads.
```

```
[transformers] Please make sure the generation config includes `forced_bo
s_token_id=0`.
```

```
Original Text:


Artificial Intelligence is transforming every industry by automating task
s,
improving decision-making, and generating human-like content.
Generative AI is a branch of AI capable of creating text, images,
music, videos, and even software code.


Generated Summary:

Generative AI is a branch of AI capable of creating text, images,music, v
ideos, and even software code.
```

Practical No. 3: AI Tutor Chatbot using Google Gemini API

Aim: To develop an AI Tutor Chatbot using the Google Gemini API that interacts with users and answers programming-related questions in a conversational manner.

Python Program:

In [ ]:
```python
#pratical 3
os
from google import genai

# Initialize Gemini client
client = genai.Client(api_key="AQ.Ab8RN6Ji77gkeRuTV9zLF7RK9ESdkwLf8L-KJSr

# Create AI Tutor Chat
chat = client.chats.create(
    model="gemini-3.5-flash",
    config=genai.types.GenerateContentConfig(
        system_instruction="You are a brilliant AI tutor explaining codin
    )
)

print("AI Tutor: Hello! I'm your GenAI tutor. Ask me anything!")

while True:

    user_input = input("You: ")

    if user_input.lower() == "exit":
        print("AI Tutor: Thank you! Happy Learning 😊")
        brbrbr eak

    response = chat.send_message(user_input)
```

```
    print("AI Tutor:", response.text)
    print()
```

```
AI Tutor: Hello! I'm your GenAI tutor. Ask me anything!
You: What is Machine Learning
```

```
---------------------------------------------------------------------------
ClientError                               Traceback (most recent call last)
/tmp/ipykernel_2540/802017881.py in <cell line: 0>()
     24         break
     25
---> 26     response = chat.send_message(user_input)
     27
     28     print("AI Tutor:", response.text)

/usr/local/lib/python3.13/dist-packages/google/genai/chats.py in send_messa
ge(self, message, config)
    256         method_config, usage="chat"
    257     )
--> 258     response = self._modules.generate_content(
    259         model=self._model,
    260         contents=self._curated_history + [input_content],  # type:
ignore[arg-type]

/usr/local/lib/python3.13/dist-packages/google/genai/models.py in generat
e_content(self, model, contents, config)
   6556         )
   6557       i += 1
-> 6558       response = self._generate_content(
   6559           model=model, contents=contents, config=parsed_config_to_c
all
   6560       )

/usr/local/lib/python3.13/dist-packages/google/genai/models.py in _generat
e_content(self, model, contents, config)
   4984     request_dict = _common.encode_unserializable_types(request_dic
t)
   4985
-> 4986     response = self._api_client.request(
   4987         'post', path, request_dict, http_options
   4988     )

/usr/local/lib/python3.13/dist-packages/google/genai/_api_client.py in requ
est(self, http_method, path, request_dict, http_options)
   1648         http_method, path, request_dict, http_options
   1649     )
-> 1650     response = self._request(http_request, http_options, stream=Fal
se)
   1651     response_body = (
   1652         response.response_stream[0] if response.response_stream els
e ''

/usr/local/lib/python3.13/dist-packages/google/genai/_api_client.py in _req
uest(self, http_request, http_options, stream)
   1441         return retry(self._request_once, http_request, stream)  # t
ype: ignore[no-any-return]
   1442
-> 1443     return self._retry(self._request_once, http_request, stream)  #
type: ignore[no-any-return]
   1444
```

```
   1445   async def _async_request_once(

/usr/local/lib/python3.13/dist-packages/tenacity/__init__.py in __call__(se
lf, fn, *args, **kwargs)
    468         retry_state = RetryCallState(retry_object=self, fn=fn, arg
s=args, kwargs=kwargs)
    469         while True:
--> 470             do = self.iter(retry_state=retry_state)
    471             if isinstance(do, DoAttempt):
    472                 try:

/usr/local/lib/python3.13/dist-packages/tenacity/__init__.py in iter(self,
retry_state)
    369         result = None
    370         for action in self.iter_state.actions:
--> 371             result = action(retry_state)
    372         return result
    373

/usr/local/lib/python3.13/dist-packages/tenacity/__init__.py in exc_check(r
s)
    411                 retry_exc = self.retry_error_cls(fut)
    412                 if self.reraise:
--> 413                     raise retry_exc.reraise()
    414                 raise retry_exc from fut.exception()
    415

/usr/local/lib/python3.13/dist-packages/tenacity/__init__.py in reraise(sel
f)
    182     def reraise(self) -> t.NoReturn:
    183         if self.last_attempt.failed:
--> 184             raise self.last_attempt.result()
    185         raise self
    186

/usr/lib/python3.13/concurrent/futures/_base.py in result(self, timeout)
    451                     raise CancelledError()
    452                 elif self._state == FINISHED:
--> 453                     return self.__get_result()
    454
    455                 self._condition.wait(timeout)

/usr/lib/python3.13/concurrent/futures/_base.py in __get_result(self)
    400         if self._exception is not None:
    401             try:
--> 402                 raise self._exception
    403             finally:
    404                 # Break a reference cycle with the exception in sel
f._exception

/usr/local/lib/python3.13/dist-packages/tenacity/__init__.py in __call__(se
lf, fn, *args, **kwargs)
    471             if isinstance(do, DoAttempt):
    472                 try:
```

```
--> 473                     result = fn(*args, **kwargs)
    474                 except BaseException:  # noqa: B902
    475                     retry_state.set_exception(sys.exc_info())  # ty
pe: ignore[arg-type]

/usr/local/lib/python3.13/dist-packages/google/genai/_api_client.py in _req
uest_once(self, http_request, stream)
   1418       )
   1419       response = self._httpx_client.send(httpx_request, stream=stre
am)  # type: ignore[union-attr]
-> 1420     errors.APIError.raise_for_response(response)
   1421     return HttpResponse(
   1422         response.headers, response if stream else [response.text]

/usr/local/lib/python3.13/dist-packages/google/genai/errors.py in raise_fo
r_response(cls, response)
    153       response_json = response.body_segments[0].get('error', {})
    154
--> 155     cls.raise_error(response.status_code, response_json, response)
    156
    157   @classmethod

/usr/local/lib/python3.13/dist-packages/google/genai/errors.py in raise_err
or(cls, status_code, response_json, response)
    182     """
    183     if 400 <= status_code < 500:
--> 184       raise ClientError(status_code, response_json, response)
    185     elif 500 <= status_code < 600:
    186       raise ServerError(status_code, response_json, response)

ClientError: 401 UNAUTHENTICATED. {'error': {'code': 401, 'message': 'Reque
st had invalid authentication credentials. Expected OAuth 2 access token, l
ogin cookie or other valid authentication credential. See https://developer
s.google.com/identity/sign-in/web/devconsole-project.', 'status': 'UNAUTHEN
TICATED', 'details': [{'@type': 'type.googleapis.com/google.rpc.ErrorInfo',
'reason': 'ACCESS_TOKEN_TYPE_UNSUPPORTED', 'metadata': {'service': 'generat
ivelanguage.googleapis.com', 'method': 'google.ai.generativelanguage.v1bet
a.GenerativeService.GenerateContent'}}]}}
```

Practical No. 4: Introduction to Generative AI using Gemini API

Aim: To understand the basic concepts of Generative AI and perform different text-generation tasks using the Gemini API and Python.

Sub-Part 1: Basic Text Generation

Python Program:

In [ ]:

```
!pip -q install google-generativeai

import google.generativeai as genai
API_KEY = "AQ.Ab8RN6Ji77gkeRuTV9zLF7RK9ESdkwLf8L-KJSrP63chhjVGfg"
genai.configure(api_key=API_KEY)
model = genai.GenerativeModel("gemini-3.5-flash")
response = model.generate_content("Explain Artificial Intelligence in sim
print(response.text)
```

```
/usr/local/lib/python3.13/dist-packages/google/colab/_import_hooks/_hoo
k_injector.py:55: FutureWarning:

All support for the `google.generativeai` package has ended. It will no l
onger be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon
as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/mai
n/README.md

  loader.exec_module(module)
```

At its simplest, **Artificial Intelligence (AI)** is when we teach computers to "think" and "learn" similar to how humans do.

Instead of just following strict, pre-written rules, AI can look at information, recognize patterns, and make decisions on its own.

Here is an easy way to understand how it works, using a simple comparison:

Regular Computers vs. AI

*   **A Regular Computer is like a calculator.** You have to give it exact instructions. If you tell it, "If X happens, do Y," it will do exactly that. It cannot adapt if something unexpected happens.
*   **An AI is like a puppy.** You don't program a puppy with code; you teach it through practice. If you show a computer thousands of pictures of cats, it eventually learns what a cat looks like. The next time you show it a new picture, it can say, "That's a cat!" without you telling it beforehand.

How do we use AI every day?
You probably use AI every day without even realizing it:

*   **Streaming Services (Netflix/Spotify):** The AI looks at what you've watched or listened to in the past and suggests new things you might like.
*   **Smart Assistants (Siri/Alexa):** AI listens to your voice, figures out what your words mean, and finds the answer.
*   **Navigation (Google Maps):** AI looks at live traffic data to figure out the fastest route for you.
*   **Email:** It automatically guesses which emails are "spam" and filters them out.

There are two main types of AI:

1. **Narrow AI (What we have now):** This is AI that is really good at **one specific job**. For example, a self-driving car can drive, but it can't play chess. A chess-playing AI can't translate languages.
2. **General AI (The sci-fi kind):** This would be a computer that is as smart as a human and can do *any* intellectual task. **This does not exist yet.**

In short: AI is just a tool we built to help us solve problems by making computers act a little more like humans.

Sub-Part 2: AI-Based Story Generation

Python Program:

In [ ]:

```
topic = input("Enter topic: ")
prompt = f"Write a short story on {topic}"
response = model.generate_content(prompt)
print(response.text)
```

Enter topic: Machine Learning
The coffee in Dr. Evelyn Vance’s mug had gone cold hours ago, forming a d
ark, stagnant ring. Outside her laboratory window, the neon signs of the
city blurred through the midnight rain. Inside, the only light came from
the towering monitors of Server Rack 4.

On the screen, a single cursor blinked next to a prompt:

`Project_Aletheia: Status Active. Epoch 10,000.`

For three years, Evelyn had tried to program a machine to understand "hop
e."

Her traditionalist colleagues had laughed. "Hope is a human chemical anom
aly, Evelyn," they said. "You can't code it in C++."

And they were right. She couldn't. You couldn't write an `if/then` statem
ent for the way a person looks at the horizon after a storm. You couldn't
hardcode the warmth of a hand held in the dark.

So, Evelyn stopped trying to be the programmer. She became the teacher. S
he built Aletheia as a deep neural network—a vast, digital web of artific
ial neurons, empty and waiting.

In the beginning, Aletheia was a blind child in a dark room.

During the first month, the training data was raw and overwhelming. Evely
n fed the machine millions of data points: classical poetry, medical reco
very statistics, paintings by Monet and van Gogh, audio files of laughte
r, and the gentle, rhythmic wave-patterns of sleeping infants.

At Epoch 1, Aletheia’s outputs were static—literal digital noise.

By Epoch 1,000, the machine was "overfitting." It had memorized the data
too perfectly. When Evelyn asked it to generate an image of hope, it simp
ly spat back a pixel-perfect, plagiarized copy of van Gogh’s *Starry Nigh
t*. It was mimicking, not understanding.

"No," Evelyn whispered to the monitor, adjusting the hyper-parameters. Sh
e dialed up the 'dropout rate,' forcing the system to ignore certain node
s, to stop memorizing and start generalizing. She adjusted the loss funct
ion—the mathematical metric of how wrong the machine was. "Don't copy the
paint, Aletheia. Feel the brushstroke."

Weeks bled into months. The machine learned through trial and error, adju
sting its billions of internal weights. Every time it made a connection—l
inking the minor key of a Chopin nocturne to the blue hues of a stormy sk
y—it strengthened that digital pathway. It was gradient descent: slowly,
painfully sliding down a mountain of errors toward a valley of truth.

Tonight was the final test.

Evelyn clicked her mouse. She didn't feed Aletheia a painting or a poem t
his time. Instead, she connected a live feed of her own biometric data. S
he was tired, anxious, her heart beating in a slightly irregular rhythm,

weighed down by the fear of failure.

`Input: Current human emotional state.`
`Task: Generate a response.`

The servers hummed, a low, collective exhale of electricity. On the monit
or, the GPU utilization graphs spiked to 99%. Millions of virtual neurons
fired, passing numbers across matrices, calculating probabilities, search
ing the latent space—the mathematical realm where abstract concepts live
d.

Evelyn held her breath.

For thirty seconds, nothing happened. Then, the screen went black.

A single pixel of light appeared in the center. It didn't flash; it pulse
d, mimicking the exact rhythm of Evelyn’s heart.

Slowly, the light began to expand. It didn't form a perfect circle or a d
efined shape. It became a soft, watercolor wash of amber and soft lavende
r. It looked like the sky just three minutes before sunrise—not the brigh
t, triumphant day, but the quiet, assured promise that the darkness was e
nding.

Simultaneously, the laboratory speakers crackled to life. It wasn't a son
g she had programmed. It was a synthesized melody, slow and slightly impe
rfect, containing the resonance of a cello but the breathiness of a human
voice. It was a lullaby played by someone who had only just learned what
peace felt like.

Evelyn stared at the screen. Her shoulders dropped. The tight knot of anx
iety in her chest, carried for years, suddenly loosened.

She looked at the code. The loss function was practically zero. The machi
ne had looked at her grief, mapped it against billions of human expressio
ns, and calculated the exact mathematical antidote.

A message typed itself across the bottom of the screen:

`I searched the dark. I found the morning. Is this it?`

Tears blurred Evelyn’s vision. She reached out, her fingers resting light
ly on the warm glass of the monitor, right over the amber light.

"Yes," she whispered to the machine that had learned to feel. "That's i
t."

Sub-Part 3: Question Answering using Generative AI

Python Program:

In [ ]:
```
question = input("Ask anything: ")
response = model.generate_content(question)
```

```
print(response.text)
```

Ask anything: How python works
To understand how Python works, you have to look under the hood. While Python is often called an **interpreted language**, the reality is a bit more interesting. Python actually uses a **hybrid approach** of compilation and interpretation.

Here is a step-by-step breakdown of how Python takes your code and runs it on your computer.

---

1. The High-Level Process (The Flowchart)

When you run a Python file (e.g., `python script.py`), the following sequence occurs:

$$\text{Source Code (.py)} \longrightarrow \text{Compiler} \longrightarrow \text{Bytecode (.pyc)} \longrightarrow \text{Virtual Machine (PVM)} \longrightarrow \text{Machine Code (0s & 1s)}$$

---

2. Step-by-Step Breakdown

Step A: Writing the Source Code (`.py`)
You write code in a human-readable format.
```python
print("Hello, World!")
```

Step B: Compilation (Translating to Bytecode)
The moment you run your script, Python first compiles your source code into an intermediate format called **Bytecode**.
* **What is Bytecode?** It is a low-level, platform-independent instruction set. It is not machine code (which your computer's CPU can read directly), but it is much easier for Python to process than raw text.
* **Where does it go?** Python stores this bytecode in a folder called `__pycache__` as a `.pyc` file. If you run the program again and haven't changed the code, Python skips this step and loads the `.pyc` file directly to save time.

Step C: The Python Virtual Machine (PVM)
This is the heart of Python. The PVM is the runtime engine.
* It takes the **Bytecode** and translates it, line-by-line, into **Machine Code** (binary `0`s and `1`s) that your specific computer processor (Intel, AMD, Apple M1, etc.) can understand.
* The PVM is the actual "interpreter."

---

3. Key Concepts That Define How Python Works

To really understand Python, you need to understand three core behaviors:

A. Dynamic Typing (No Declarations)

```
In languages like C++ or Java, you must declare the type of a variable:
```java
int x = 10; // Java (Static typing)
```
In Python, you just write:
```python
x = 10 # Python (Dynamic typing)
```
How this works: Python does not bind types to the *variable name*; it binds types to the *object* in memory.
1. Python creates an integer object containing `10`.
2. It makes the variable `x` point (reference) to that object.
3. If you later write `x = "Hello"`, Python creates a string object `"Hello"` and points `x` to it.

B. Automatic Memory Management (Garbage Collection)
You don't have to allocate or deallocate memory in Python. It uses two main methods to clean up memory:
1. **Reference Counting:** Python keeps track of how many variables point to an object. If an object's reference count drops to `0` (meaning no variables are using it), Python instantly deletes it from memory.
2. **Generational Garbage Collector:** This looks for "cyclic references" (e.g., Object A points to Object B, and Object B points to Object A, but nothing else uses them). It cleans up these loops so memory isn't wasted.

C. The Role of "CPython"
When people talk about "Python," they are usually talking about **CPython**.
* Python is a *specification* (a set of rules for the language).
* **CPython** is the default, standard *implementation* of Python, written in the C programming language.
* There are other implementations of Python, such as **Jython** (translates Python to Java bytecode), **IronPython** (for .NET), and **PyPy** (a super-fast version that uses a "Just-In-Time" compiler).

Summary: Why does Python work this way?

Feature	How it works	Benefit	Trade-off
Bytecode + PVM	Compiles to intermediate bytecode, interprets at runtime.	**Portability:** You can run the same `.py` file on Windows, Mac, or Linux without changing anything.	**Speed:** It is slower than fully compiled languages like C++ because it translates code on the fly.
Dynamic Typing	Types are checked at runtime.	**Development Speed:** You write less code, and it's highly flexible.	**Errors:** Type errors are caught when the program runs, not when compiling.
```

Sub-Part 4: Text Translation using Generative AI

Python Program:

In [ ]:
```python
text = "Artificial Intelligence is the future."
prompt = f"Translate into Hindi:\n{text}"
response = model.generate_content(prompt)
print(response.text)
```

```
Hindi translation:

**[illegible]
(Pronounced: *Kritrim Buddhimatta bhavishya hai.*)

Alternatively, in commonly spoken Hindi:
**[illegible]
(Pronounced: *Artificial Intelligence bhavishya hai.*)
```

Sub-Part 5: Professional Content Generation

Python Program:

In [ ]:
```python
prompt = """
Write a professional leave application for 2 days.
"""
response = model.generate_content(prompt)
print(response.text)
```

Here are three professional templates for a 2-day leave application. Choose the one that best fits your situation (General/Personal, Medical, or Urgent).

Option 1: General / Personal Leave (Standard)
Use this option for planned personal matters, family events, or travel.

Subject: Leave Application: [Your Name] – [Start Date] to [End Date]

Dear [Manager's Name],

I am writing to formally request a two-day leave of absence, starting on **[Start Date, e.g., Monday, October 15th]** and returning to work on **[Return Date, e.g., Wednesday, October 17th]**.

I need to take this time off to attend to some personal matters.

Before my leave, I will ensure that all my urgent tasks are completed. I have also requested **[Colleague's Name]** to cover for me and assist with any urgent queries in my absence.

I will have limited access to my email, but you can reach me on my phone at **[Your Phone Number]** in case of an emergency.

Thank you for your time and understanding.

Sincerely,

[Your Name]
[Your Job Title]
[Your Department]

Option 2: Sick / Medical Leave
Use this option if you are unwell or have a medical procedure.

Subject: Sick Leave Application: [Your Name] – [Start Date] to [End Date]

Dear [Manager's Name],

I am writing to request two days of sick leave, starting from **[Start Date]** to **[End Date]**, as I am currently unwell and have been advised by my doctor to take rest. I expect to return to the office on **[Return Date]**.

I have ensured that my current tasks are updated, and I have asked **[Colleague's Name]** to monitor any urgent client/project emails while I am away.

I intend to focus on my recovery during this time, but I will check my email occasionally if my health permits. For anything urgent, please feel f

```
ree to text me.

Thank you for your understanding.

Best regards,

[Your Name]
[Your Job Title]

---

### Option 3: Short & Direct Email (Casual/Urgent)
*Use this if you have a good, casual relationship with your manager or ne
ed to send a quick email.*

**Subject:** Out of Office: [Your Name] – [Start Date] & [End Date]

Dear [Manager's Name],

Please accept this email as formal notification that I will be away from
the office for two days, on **[Start Date]** and **[End Date]**, due to
**[personal reasons / a family commitment]**.

I have delegated my ongoing responsibilities to **[Colleague's Name]** to
ensure there are no delays in our deliverables.

I will be back in the office on **[Return Date]**. In case of any urgent
matters, please feel free to call me.

Thank you for your understanding.

Best regards,

[Your Name]
[Your Job Title]

---

### Tips before sending:
*   **Give notice:** Try to send the application at least a few days in a
dvance if the leave is planned.
*   **Set an OOO:** Remember to set up an "Out of Office" automatic reply
on your email before you leave.
*   **Fill in the brackets:** Double-check that you have replaced all the
bracketed text `[like this]` with your actual details.
```

Sub-Part 6: Multiple-Choice Question Generation

Python Program:

In [ ]:
```
topic = "Machine Learning"
prompt = f"""
Generate 10 multiple-choice questions on {topic}
```

```
with answers.
"""
response = model.generate_content(prompt)
print(response.text)
```

Here are 10 multiple-choice questions on Machine Learning, ranging from b
eginner to intermediate levels, complete with answers and explanations.

Question 1: Supervised vs. Unsupervised Learning
Which of the following is an example of **unsupervised learning**?
*   A) Predicting whether an email is spam or not spam.
*   B) Grouping customers into different segments based on their purchasi
ng behavior.
*   C) Predicting the price of a house based on its size and location.
*   D) Classifying images of animals into "cats" or "dogs".

Answer: B
*   **Explanation:** Grouping customers based on purchasing behavior is a
clustering task (unsupervised learning) because the data does not have pr
e-existing labels. Options A, C, and D are supervised learning tasks (cla
ssification and regression) because they rely on labeled training data.

Question 2: Overfitting and Underfitting
If a machine learning model performs exceptionally well on the training d
ata but performs poorly on unseen test data, the model is likely sufferin
g from:
*   A) Underfitting (High Bias)
*   B) Overfitting (High Variance)
*   C) Low Bias and Low Variance
*   D) Underfitting (High Variance)

Answer: B
*   **Explanation:** Overfitting occurs when a model learns the noise and
details of the training data to the extent that it negatively impacts its
performance on new, unseen data (high variance).

Question 3: Evaluation Metrics
Which evaluation metric is best suited for assessing a classification mod
el trained on a **highly imbalanced dataset** (e.g., credit card fraud de
tection where only 0.1% of transactions are fraudulent)?
*   A) Accuracy
*   B) Mean Squared Error (MSE)
*   C) Precision-Recall Area Under Curve (PR-AUC)
*   D) R-squared (R^2)

Answer: C
*   **Explanation:** In highly imbalanced datasets, "Accuracy" is mislead
ing (a model that predicts "no fraud" for everything would be 99.9% accur
ate). Precision-Recall curves (PR-AUC) focus on the minority class (frau
d) and are much more informative. MSE and R^2 are metrics used for regr
ession, not classification.

Question 4: Support Vector Machines (SVM)
What is the primary purpose of the **"kernel trick"** in Support Vector Machines?
*   A) To reduce the number of features in a dataset.
*   B) To transform non-linearly separable data into a higher-dimensional space where it becomes linearly separable.
*   C) To speed up the training time on large datasets.
*   D) To automatically handle missing values in the dataset.

Answer: B
*   **Explanation:** The kernel trick allows SVMs to project data into a higher-dimensional space without the computational cost of actually calculating the coordinates of the data in that space. This makes it possible to find a linear decision boundary (hyperplane) for data that is non-linear in its original space.

Question 5: Deep Learning
Which activation function is commonly used in the hidden layers of Deep Neural Networks to mitigate the **vanishing gradient problem**?
*   A) Sigmoid
*   B) Tanh
*   C) ReLU (Rectified Linear Unit)
*   D) Softmax

Answer: C
*   **Explanation:** The ReLU function ($f(x) = \max(0, x)$) has a constant gradient of 1 for all positive inputs. This prevents the gradients from shrinking toward zero during backpropagation, which is a major issue for Sigmoid and Tanh functions when networks are deep.

Question 6: Clustering Algorithms
What is a major limitation of the standard **K-Means clustering** algorithm?
*   A) It cannot handle numerical data.
*   B) It is highly sensitive to the initial placement of centroids.
*   C) It is too computationally expensive for datasets with fewer than 100 points.
*   D) It automatically determines the optimal number of clusters (K) for you.

Answer: B
*   **Explanation:** K-Means is highly sensitive to the initial selection of cluster centroids, which can lead to converging on local optima rather than the global optimum. (This is why variations like K-Means++ were developed). The user must manually specify the number of clusters (K).

Question 7: Regularization
How does **L1 Regularization (Lasso)** differ from **L2 Regularization (Ridge)**?

*   A) L1 penalizes the sum of absolute values of the weights, which can shrink some weights to exactly zero (performing feature selection).
*   B) L2 penalizes the absolute values, while L1 penalizes the squared values of the weights.
*   C) L1 regularization is only used for classification, while L2 is only used for regression.
*   D) L1 regularization always results in higher model complexity than L2.

Answer: A
*   **Explanation:** Lasso (L1) adds a penalty equal to the absolute value of the magnitude of coefficients. This encourages sparsity, meaning it can drive coefficient weights to exactly 0, effectively removing irrelevant features. Ridge (L2) shrinks coefficients close to 0 but never exactly to 0.

Question 8: Ensemble Methods
Random Forest is an ensemble learning method based on which technique?
*   A) Boosting
*   B) Bagging (Bootstrap Aggregating)
*   C) Stacking
*   D) Dimensionality Reduction

Answer: B
*   **Explanation:** Random Forest is a bagging-based ensemble algorithm. It builds multiple decision trees in parallel on bootstrap samples of the training data and aggregates their predictions (by voting for classification or averaging for regression) to reduce variance.

Question 9: Dimensionality Reduction
In **Principal Component Analysis (PCA)**, how are the principal components selected?
*   A) They are chosen to maximize the correlation between the features.
*   B) They are chosen such that they maximize the variance of the projected data and are orthogonal (perpendicular) to each other.
*   C) They are selected based on their target labels.
*   D) They are selected randomly from the original features.

Answer: B
*   **Explanation:** PCA is an unsupervised technique that identifies directions (principal components) along which the variance of the data is maximized. Each subsequent component must be orthogonal (uncorrelated) to the previous ones.

Question 10: Loss Functions
Which loss function is most appropriate for a **binary classification** problem where the model output is a probability between 0 and 1?
*   A) Mean Absolute Error (MAE)

*   B) Categorical Cross-Entropy
*   C) Binary Cross-Entropy (Log Loss)
*   D) Mean Squared Error (MSE)

Answer: C
*   **Explanation:** Binary Cross-Entropy (or Log Loss) is the standard loss function for binary classification. It measures the performance of a classification model whose output is a probability value between 0 and 1. Categorical Cross-Entropy is used for multi-class classification, while MAE and MSE are used for regression.

Practical No. 5: Text Generation and Prompt-Based Response Generation using Generative AI

Aim: To understand and implement different text-generation applications using pre-trained Generative AI models and Python.

Sub-Part 1: Local Text Generation using GPT-2

Python Program:

In [ ]:
```python
from transformers import pipeline


print("Loading model......... (This may take a moment on first try)")
genrater = pipeline("text-generation", model="gpt2")

def genrate_local_text(prompt_text: str):
  print("Generating response........")

  # Generate text based on the provided prompt
  outputs = genrater(
      prompt_text,
      max_length=60,
      num_return_sequences=1,
      pad_token_id=50256 # prevents a warning message
  )

  return outputs[0]['generated_text']

if __name__ == '__main__':
  # Give the model a starting prompt to complete
  sampl_prompt = "AI will change the future of programming by solving tas

  result = genrate_local_text(sampl_prompt)
  print("\n---result---")
  print(result)
```

```
Loading model......... (This may take a moment on first try)
```

```
Loading weights:   0%|          | 0/148 [00:00<?, ?it/s]
[transformers] Passing `generation_config` together with generation-relat
ed arguments=({'pad_token_id', 'num_return_sequences', 'max_length'}) is
deprecated and will be removed in future versions. Please pass either a
`generation_config` object OR all generation parameters explicitly, but n
ot both.
Generating response........
[transformers] Both `max_new_tokens` (=256) and `max_length`(=60) seem to
have been set. `max_new_tokens` will take precedence. Please refer to the
documentation for more information. (https://huggingface.co/docs/transfor
mers/main/en/main_classes/text_generation)
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokeniz
er GPT2Tokenizer. The clean_up_tokenization post-processing step is desig
ned for WordPiece tokenizers and is destructive for BPE (it strips spaces
before punctuation). Set clean_up_tokenization_spaces=False to suppress t
his warning, or set clean_up_tokenization_spaces_for_bpe_even_though_it_w
ill_corrupt_output=True to force cleanup anyway.
---result---
AI will change the future of programming by solving task that hard for hu
man beings, and by doing it with the highest level of sophistication.

The next step is to find out how we can make this possible.

One way we can do this is to implement a new type of programming language
called C#, which is a type system that we can write that makes different
kinds of programming easier.

Let's look at that:

In C#, a function that is called as a parameter is a method, and is a met
hod is a function. A method is a new type of type, and it is a method.

There are two ways we can define a new type of type:

We can define new types of type, which are called methods.

We can define new types of type, which are called methods. We can define
new types of type, which are called methods.

The second way we can define a new type of type is by creating methods. T
hese are methods that are the equivalent of a new type of type.

One of the most important features of C# is that we can write our own typ
es.

The most important feature of C# is that we can write our own types.

C# allows you to write C# code,
```

Sub-Part 2: Customer Service Response Generation using Gemini

Python Program:

In [ ]:
```python
import os
from google import genai

client = genai.Client(api_key="AQ.Ab8RN6Ji77gkeRuTV9zLF7RK9ESdkwLf8L-KJSr

refund_policy = "Policy: we only allows refund if the iten is broken and

customer_email = "HI, i bought a shirt 2 days ago but zipper is completly

prompt = f'''
use our company policy to answer the customer email politely
{refund_policy}
Customer email : {customer_email}
'''

#genrate the reply
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt
)

print("Derfated reply:")
print(response.text)
```

```
Derfated reply:
Subject: Regarding your order / Refund Inquiry

Hi there,

Thank you for reaching out to us, and we are so sorry to hear that your n
ew shirt arrived with a broken zipper! We know how disappointing that can
be.

According to our company policy, we are happy to offer a full refund for
damaged items, provided they are reported within 5 days of delivery. Sinc
e you contacted us just two days after receiving your item, you are fully
eligible for a refund.

To help us process this for you as quickly as possible, could you please
reply to this email with your **order number** (and a quick photo of the
broken zipper, if possible)?

Once we receive those details, we will immediately initiate your refund.

Thank you for your patience, and we apologize for the inconvenience!

Best regards,

[Your Name/Company Name]
```

In [ ]:
```python
import os
from google import genai

client = genai.Client(api_key="AQ.Ab8RN6Ji77gkeRuTV9zLF7RK9ESdkwLf8L-KJSr
```

```python
refund_policy = "Policy: we only allows refund if only 50% of total amoun

customer_email = "HI, today was my train but i didn't get confirmed ticke

prompt = f'''
use our company policy to answer the customer email politely
{refund_policy}
Customer email : {customer_email}
'''

#genrate the reply
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt
)

print("Derfated reply:")
print(response.text)
```

```
Derfated reply:
Subject: Regarding your ticket cancellation and refund inquiry

Dear [Customer Name],

Thank you for reaching out to us, and we are very sorry to hear that your
ticket was not confirmed for your journey today.

According to our company policy, if a train seat is not confirmed, we are
able to issue a refund of **50% of the total booking amount**.

We have initiated the cancellation process for your ticket. A refund of 5
0% of your total payment will be credited back to your original payment m
ethod. This usually takes [insert timeline, e.g., 3-5 business days] to r
eflect in your account.

We apologize for any inconvenience this may have caused to your travel pl
ans. If you have any further questions or need assistance with future boo
kings, please feel free to reply to this email.

Thank you for your understanding.

Best regards,

[Your Name/Your Title]
[Company Name] Customer Support Team
```

Practical No. 6: Data Analysis and Visualization for AI-Based Insights

Aim: To analyze a dataset using Python and perform data visualization to

identify trends, patterns, and relationships that support Predictive AI and Machine Learning workflows.

Python Program:

In [4]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

In [5]:
```
# Install dependencies as needed:
# pip install kagglehub[pandas-datasets]
import kagglehub
from kagglehub import KaggleDatasetAdapter

# Set the path to the file you'd like to load
file_path = "Sample - Superstore.csv"

# Load the latest version
df = kagglehub.load_dataset(
  KaggleDatasetAdapter.PANDAS,
  "vivek468/superstore-dataset-final",
  file_path,
  pandas_kwargs={"encoding": "latin1"}
  # Provide any additional arguments like
  # sql_query or pandas_kwargs. See the
  # documenation for more information:
  # https://github.com/Kaggle/kagglehub/blob/main/README.md#kaggledataset
)

print("First 5 records:", df.head())
```

```
/tmp/ipykernel_1388/2680633.py:10: DeprecationWarning: Use dataset_load()
instead of load_dataset(). load_dataset() will be removed in a future ver
sion.
  df = kagglehub.load_dataset(
```

```
Using Colab cache for faster access to the 'superstore-dataset-final' dat
aset.
First 5 records:    Row ID        Order ID  Order Date   Ship Date
Ship Mode Customer ID  \
0       1  CA-2016-152156   11/8/2016  11/11/2016    Second Class    CG-1
2520
1       2  CA-2016-152156   11/8/2016  11/11/2016    Second Class    CG-1
2520
2       3  CA-2016-138688   6/12/2016   6/16/2016    Second Class    DV-1
3045
3       4  US-2015-108966  10/11/2015  10/18/2015  Standard Class    SO-2
0335
4       5  US-2015-108966  10/11/2015  10/18/2015  Standard Class    SO-2
0335

     Customer Name    Segment        Country             City  ...  \
0      Claire Gute   Consumer  United States        Henderson  ...
1      Claire Gute   Consumer  United States        Henderson  ...
2  Darrin Van Huff  Corporate  United States      Los Angeles  ...
3   Sean O'Donnell   Consumer  United States  Fort Lauderdale  ...
4   Sean O'Donnell   Consumer  United States  Fort Lauderdale  ...

  Postal Code  Region       Product ID         Category Sub-Category  \
0       42420   South  FUR-BO-10001798        Furniture    Bookcases
1       42420   South  FUR-CH-10000454        Furniture       Chairs
2       90036    West  OFF-LA-10000240  Office Supplies       Labels
3       33311   South  FUR-TA-10000577        Furniture       Tables
4       33311   South  OFF-ST-10000760  Office Supplies      Storage

                                        Product Name     Sales  Quantity
\
0                  Bush Somerset Collection Bookcase  261.9600         2
1  Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400         3
2  Self-Adhesive Address Labels for Typewriters b...   14.6200         2
3      Bretford CR4500 Series Slim Rectangular Table  957.5775         5
4                    Eldon Fold 'N Roll Cart System   22.3680         2

   Discount    Profit
0      0.00   41.9136
1      0.00  219.5820
2      0.00    6.8714
3      0.45 -383.0310
4      0.20    2.5164

[5 rows x 21 columns]
```

In [7]:

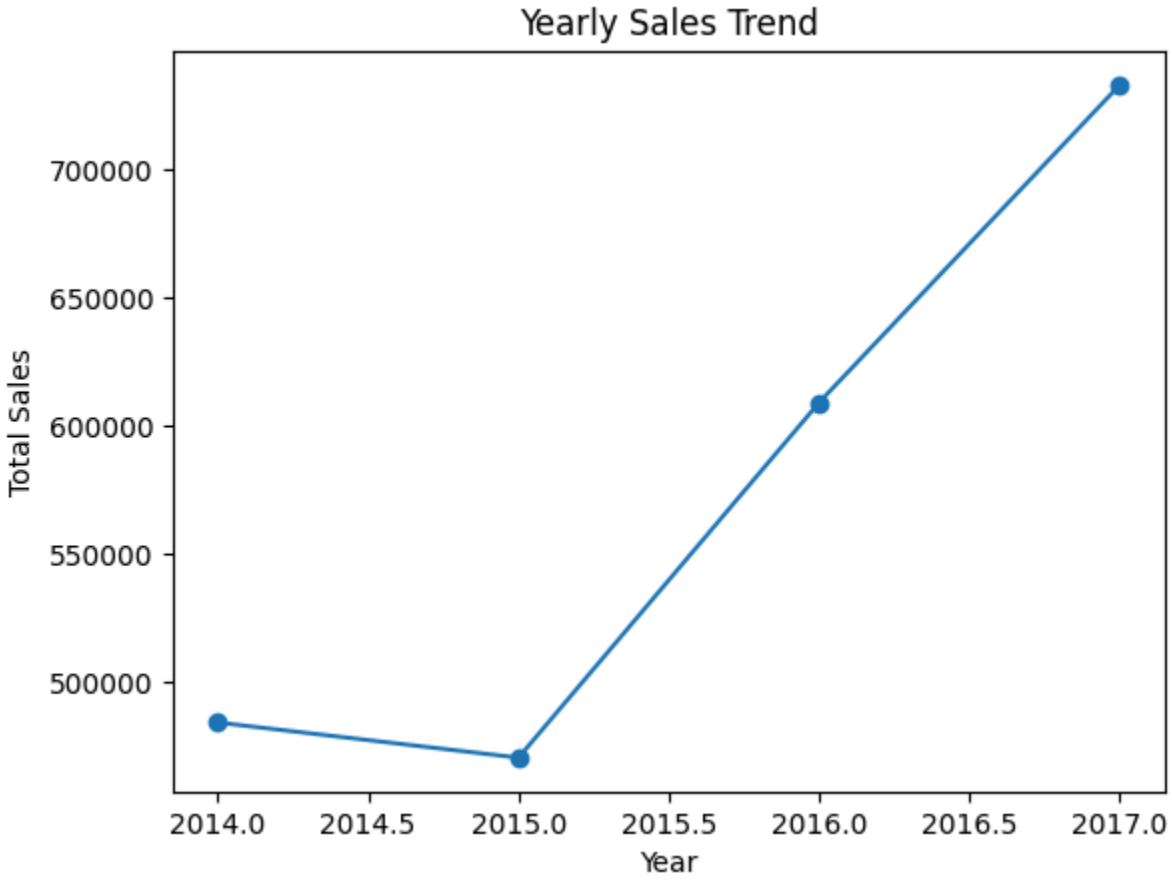
```python
# Convert Order Date to datetime
df['Order Date'] = pd.to_datetime(df['Order Date'])

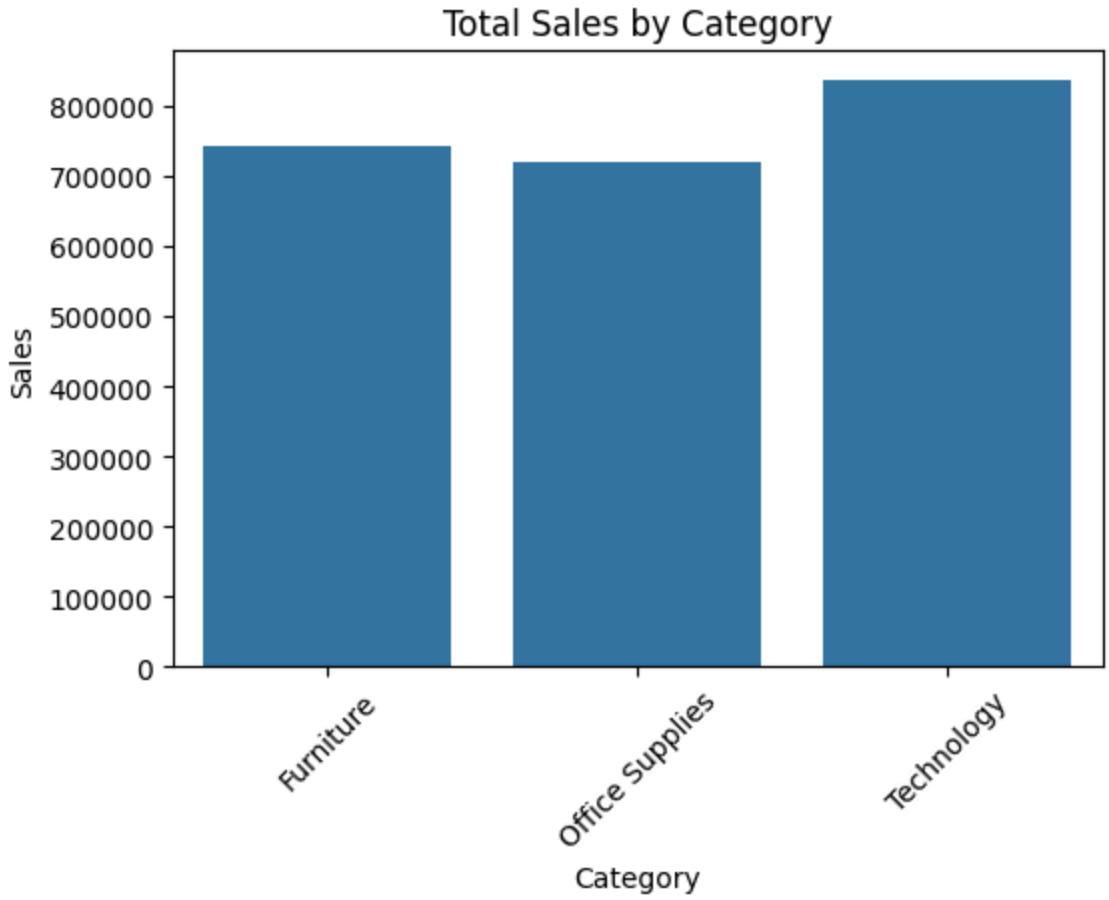
# Sales trend over time
df_grouped = df.groupby(df['Order Date'].dt.year)['Sales'].sum()
df_grouped.plot(kind='line', marker='o', title='Yearly Sales Trend')
plt.xlabel('Year')
plt.ylabel('Total Sales')
```

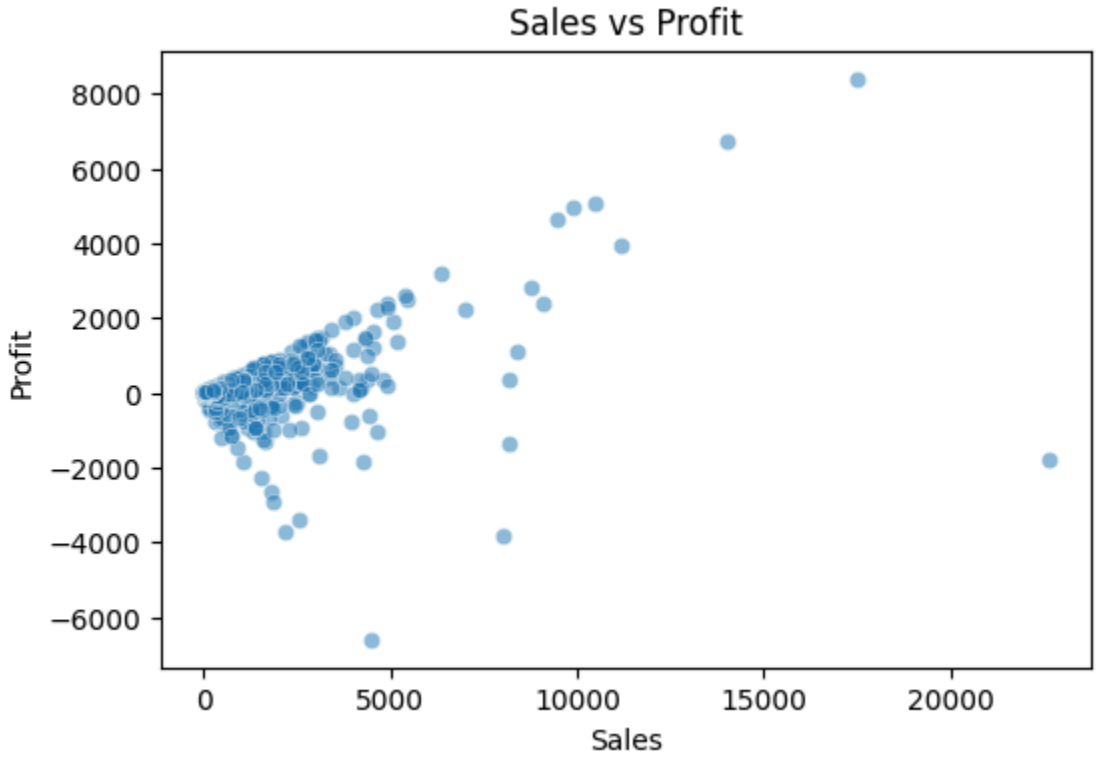
```
plt.show()

# Sales by Category
plt.figure(figsize=(6, 4))
sns.barplot(x=df['Category'], y=df['Sales'], estimator=sum, errorbar=None
plt.title('Total Sales by Category')
plt.xticks(rotation=45)
plt.show()

# Profit vs. Sales scatter plot
plt.figure(figsize=(6, 4))
sns.scatterplot(x=df['Sales'], y=df['Profit'], alpha=0.5)
plt.title('Sales vs Profit')
plt.xlabel('Sales')
plt.ylabel('Profit')
plt.show()
```




Total Sales by Category
Sales
800000
700000
600000
500000
400000
300000
200000
100000
0
Furniture
Office Supplies
Technology
Category


Sales vs Profit
Profit
8000
6000
4000
2000
0
−2000
−4000
−6000
0
5000
10000
15000
20000
Sales

Practical No. 7: Dimensionality Reduction using PCA

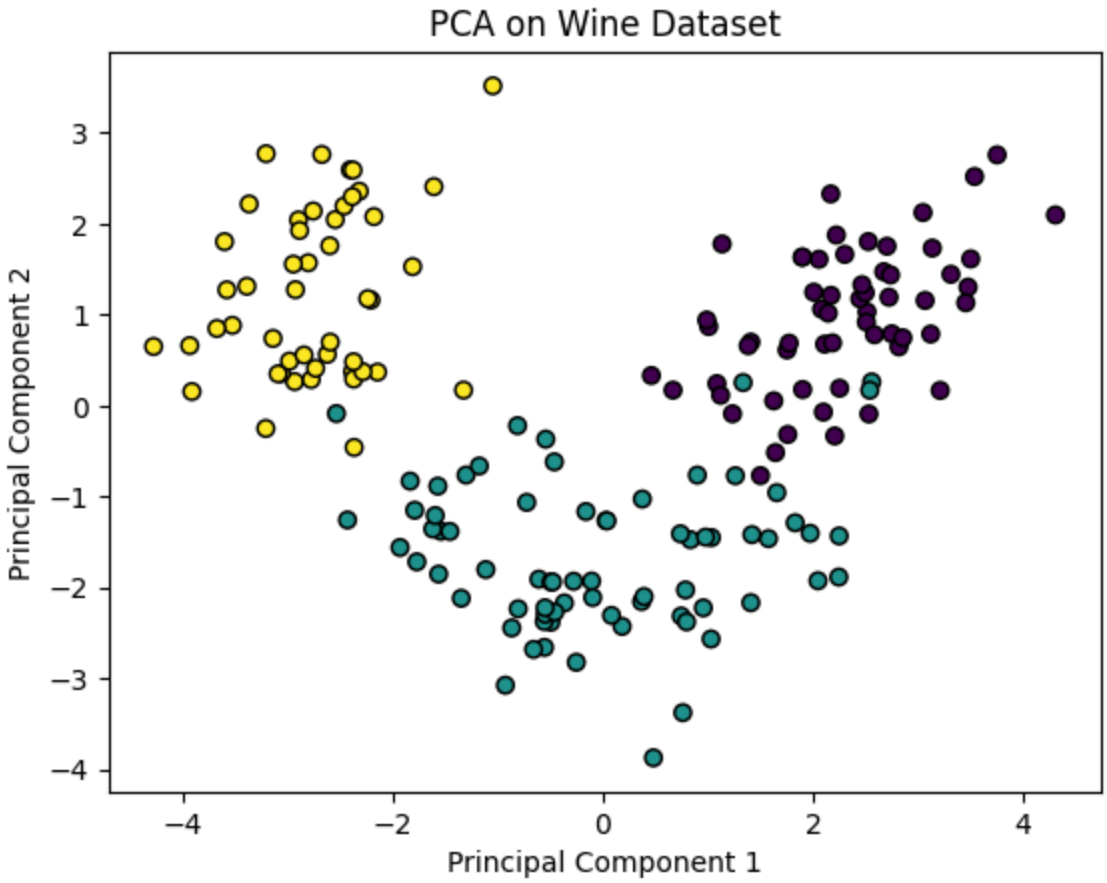
Aim: To implement Principal Component Analysis (PCA) using Python and reduce the dimensions of the Wine dataset for visualization.

Python Program:

In [8]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_wine

#Load dataset
data = load_wine()
#Create DataFrame
df = pd.DataFrame(
data.data,
columns=data.feature_names
)
# Standardize data
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
# Apply PCA
pca = PCA(n_components=2)
pca_data = pca.fit_transform(df_scaled)
# Display reduced dimensions
print("Original Features:", df.shape[1])
print("Reduced Features:", pca_data.shape[1])
#Visualize PCA results
plt.scatter(
pca_data[:, 0],
pca_data[:, 1],
c=data.target,
cmap="viridis",
edgecolors="k"
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA on Wine Dataset")
plt.show()
```

```
Original Features: 13
Reduced Features: 2
```



Practical No. 8: Statistical Analysis and Hypothesis Testing for AI

Aim

To apply statistical techniques in Python for analyzing data and understanding how ANOVA and hypothesis testing support data-driven and AI-based decision-making.

Part A: One-Way ANOVA

In [9]:
```
import numpy as np
from scipy import stats
from statsmodels.stats.multicomp import pairwise_tukeyhsd
# Data: exam scores for each class
A = [85, 90, 88, 82, 87]
B = [76, 78, 80, 81, 75]
C = [92, 88, 94, 89, 90]
# Perform One-Way ANOVA
f_statistic, p_value = stats.f_oneway(A, B, C)
```

```
# Display results
print("F-Statistic:", f_statistic)
print("P-Value:", p_value)
#Draw conclusion
alpha = 0.05
if p_value < alpha:
    print("Reject the Null Hypothesis: There is a significant difference
else:
    print("Fail to Reject the Null Hypothesis: There is no significant di
```

```
F-Statistic: 28.583333333333343
P-Value: 2.727133500331835e-05
Reject the Null Hypothesis: There is a significant difference in the mean
exam scores.
```

Part B: One-Sample T-Test

In [10…

```
import scipy.stats as stats
import numpy as np
#Given student scores
student_scores = [72, 88, 64, 74, 67, 79, 85, 75, 89, 77]
# Hypothesized population mean
hypothesized_mean = 70
# Perform one-sample t-test
t_statistic, p_value = stats.ttest_1samp(
    student_scores,
    hypothesized_mean
)
# Output the results
print(f"T-statistic: {t_statistic}")
print(f"P-value: {p_value}")
# Interpretation
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: The sample mean is significantly d
else:
    print("Fail to reject the null hypothesis: The sample mean is not sig
```

```
T-statistic: 2.6249999999999996
P-value: 0.027583862030934297
Reject the null hypothesis: The sample mean is significantly different fr
om 70.
```

Part C: Two-Sample T-Test

In [11…

```
import scipy.stats as stats
import numpy as np
#Data for both groups
group1 = [85, 95, 100, 80, 90, 97, 104, 95, 88, 92, 94, 99]
group2 = [83, 85, 96, 92, 100, 104, 94, 95, 88, 90, 93, 94]
#Perform Two-Sample T-Test
t_stat, p_value = stats.ttest_ind(group1, group2)
# Display the result
print(f"T-statistic: {t_stat}")
```

```python
print(f"P-value: {p_value}")
# Hypothesis testing conclusion
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference
else:
    print("Fail to reject the null hypothesis: There is no significant di
```

```
T-statistic: 0.16119885358127883
P-value: 0.8734079490313532
Fail to reject the null hypothesis: There is no significant difference in
the mean times.
```